Level 7: CTE & Recursive Queries

CTEs (WITH ... AS ...) give a subquery a readable name you can reuse, and chain multiple named steps together. Recursive CTEs go further, letting a query call itself to walk hierarchies of unknown depth -- the single technique behind org charts, category trees, folder structures, and bill-of-materials explosions.

Q171. Simple CTE - High earners EASY

```
WITH high_earners AS (
    SELECT name, salary, department
    FROM employees
    WHERE salary > 80000
)
SELECT * FROM high_earners;
```

WHAT THIS QUERY DOES

You've already seen CTEs used as a workaround in Level 6 (Q152/153) — here's the pattern in its simplest form. `WITH high_earners AS (...)` defines a named, temporary result set, and the query below it can then reference `high_earners` exactly as if it were a real table. For a query this simple, a CTE isn't strictly necessary (you could just filter `employees` directly) — its real value shows up once a query gets complex enough that giving an intermediate step its own name makes the whole thing easier to read, as the next few questions demonstrate.

RESULT ON OUR SAMPLE DATA

name	salary	department
Rahul Singh	85,000	Finance
Vikram Reddy	90,000	IT
Karthik Nair	95,000	IT
Arjun Reddy	82,000	IT
Suresh Babu	88,000	IT
Ravi Teja	91,000	IT

6 rows returned.

Q172. Multiple CTEs - Department stats MEDIUM

```
WITH dept_avg AS (
    SELECT department, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
```

```
),
dept_count AS (
    SELECT department, COUNT(*) AS emp_count
    FROM employees
    GROUP BY department
)
SELECT da.department, da.avg_salary, dc.emp_count
FROM dept_avg da
JOIN dept_count dc ON da.department = dc.department;
```

WHAT THIS QUERY DOES

You can define *multiple* CTEs in one `WITH` clause, separated by commas, and even join them together afterward as if they were ordinary tables. This is often clearer than nesting subqueries directly inside a JOIN's `ON` clause (as Q105 did) — each CTE gets a readable name and can be understood on its own, one piece at a time, before seeing how they're combined.

RESULT ON OUR SAMPLE DATA

department	avg_salary	emp_count
Finance	74,000	3
HR	63,000	3
IT	84,714.29	7
Marketing	63,000	2

4 rows returned.

Q173. Recursive employee hierarchy HARD

```
WITH RECURSIVE emp_hierarchy AS (
    -- Base case: top-level employees
    SELECT emp_id, name, manager_id, 0 AS level, CAST(name AS CHAR(500)) AS path
    FROM employees
    WHERE manager_id IS NULL

    UNION ALL

    -- Recursive case: direct reports
    SELECT e.emp_id, e.name, e.manager_id, h.level + 1,
           CONCAT(h.path, ' -> ', e.name)
    FROM employees e
    JOIN emp_hierarchy h ON e.manager_id = h.emp_id
)
SELECT * FROM emp_hierarchy ORDER BY level, emp_id;
```

WHAT THIS QUERY DOES

Your first **recursive CTE** — a CTE that references *itself*, used whenever data has an unknown, variable depth (an org chart can be 2 levels deep or 10; you don't know in

advance). It always has two parts joined by `UNION ALL`: a **base case** (here, employees with no manager — the top of the tree, at level 0) and a **recursive case** (employees whose manager was found in the *previous* round, each one level deeper than their manager). SQL keeps re-running the recursive part, feeding each round's results back in as input to the next, until a round produces no new rows — at which point it stops automatically. The `path` column, built up with string concatenation at each step, is a nice bonus that shows the full management chain down to each employee.

RESULT ON OUR SAMPLE DATA

emp_id	name	manager_id	level	path
1	Aarav Sharma	*NULL*	0	Aarav Sharma
2	Priya Patel	1	1	Aarav Sharma -> Priya Patel
3	Rahul Singh	1	1	Aarav Sharma -> Rahul Singh
4	Neha Gupta	1	1	Aarav Sharma -> Neha Gupta
5	Vikram Reddy	1	1	Aarav Sharma -> Vikram Reddy
7	Karthik Nair	1	1	Aarav Sharma -> Karthik Nair
15	Ravi Teja	1	1	Aarav Sharma -> Ravi Teja
6	Ananya Menon	2	2	Aarav Sharma -> Priya Patel -> Ananya Menon
8	Meera Krishnan	5	2	Aarav Sharma -> Vikram Reddy -> Meera Krishnan
9	Arjun Reddy	5	2	Aarav Sharma -> Vikram Reddy -> Arjun Reddy
11	Rajesh Kumar	3	2	Aarav Sharma -> Rahul Singh -> Rajesh Kumar
12	Deepa Iyer	2	2	Aarav Sharma -> Priya Patel -> Deepa Iyer
13	Suresh Babu	5	2	Aarav Sharma -> Vikram Reddy -> Suresh Babu
14	Kavya Nair	3	2	Aarav Sharma -> Rahul Singh -> Kavya Nair
10	Sneha Patel	8	3	Aarav Sharma -> Vikram Reddy -> Meera Krishnan -> Sneha Patel

15 rows returned.

Q174. Recursive category hierarchy MEDIUM

```
-- Example: Product categories with parent-child
CREATE TABLE categories (
    cat_id INT PRIMARY KEY,
    cat_name VARCHAR(100),
    parent_id INT
);

WITH RECURSIVE category_tree AS (
    SELECT cat_id, cat_name, parent_id, 0 AS level, cat_name AS path
    FROM categories
```

```
    WHERE parent_id IS NULL

    UNION ALL

    SELECT c.cat_id, c.cat_name, c.parent_id, ct.level + 1,
           CONCAT(ct.path, ' > ', c.cat_name)
    FROM categories c
    JOIN category_tree ct ON c.parent_id = ct.cat_id
)
SELECT * FROM category_tree ORDER BY level, cat_id;
```

WHAT THIS QUERY DOES

An illustrative `categories` table has been added (Electronics > Computers > Laptops > Gaming Laptops, plus a separate Accessories and Software branch) to demonstrate this concretely. Structurally, this is the exact same recursive-CTE template as Q173's employee hierarchy — a self-referencing `parent_id` column, a base case for top-level rows, and a recursive case joining each round's results back to their children. Once you recognize this shape, it applies to *any* parent-child hierarchy: org charts, product categories, folder trees (Q180), threaded comments, and more — the table changes, but the recursive-CTE skeleton stays identical.

RESULT ON OUR SAMPLE DATA

cat_id	cat_name	parent_id	level	path
1	Electronics	*NULL*	0	Electronics
6	Software	*NULL*	0	Software
2	Computers	1	1	Electronics > Computers
5	Accessories	1	1	Electronics > Accessories
3	Laptops	2	2	Electronics > Computers > Laptops
4	Gaming Laptops	3	3	Electronics > Computers > Laptops > Gaming Laptops

6 rows returned.

Q175. Generate numbers 1 to 100 MEDIUM

```
WITH RECURSIVE numbers AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1 FROM numbers WHERE n < 100
)
SELECT * FROM numbers;
```

WHAT THIS QUERY DOES

A recursive CTE doesn't have to reference a real table at all — here the base case is just the literal number 1, and the recursive case adds 1 each round, stopping once n reaches 100. This 'number generator' pattern is genuinely useful in practice: it's a common way to fill in gaps in a report (e.g. showing '0 orders' for days that had no

orders at all, which a plain GROUP BY would simply omit) by generating every possible number/date first and then LEFT JOINing your real data onto it.

RESULT ON OUR SAMPLE DATA

n
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20

Showing first 20 of 100 rows.

Q176. Calendar generation for 2024 MEDIUM

```
WITH RECURSIVE calendar AS (
    SELECT '2024-01-01' AS date
    UNION ALL
    SELECT DATE_ADD(date, INTERVAL 1 DAY)
    FROM calendar
    WHERE date < '2024-12-31'
)
SELECT date, DAYNAME(date) AS day_of_week FROM calendar;
```

WHAT THIS QUERY DOES

The same number-generator idea from Q175, applied to dates instead of integers: start at Jan 1st, and each recursive round adds one more day, until the end of the

year is reached. Generating a full calendar table like this is the standard trick for reports that need every date to appear (even ones with zero activity) rather than only the dates that happen to already exist in your data. *(The live example below generates just the first 10 days rather than all 365, purely to keep the printed table a reasonable size — the technique is identical either way. `DATE_ADD`/`DAYNAME` are MySQL functions; SQLite's equivalents are `date(d, '+1 day')` and `strftime('%w', d)`, the latter returning a weekday number 0-6 rather than a name.)*

RESULT ON OUR SAMPLE DATA

date	day_of_week
2024-01-01	1
2024-01-02	2
2024-01-03	3
2024-01-04	4
2024-01-05	5
2024-01-06	6
2024-01-07	0
2024-01-08	1
2024-01-09	2
2024-01-10	3

10 rows returned.

Q177. Running balance with CTE MEDIUM

```
WITH transactions AS (
    SELECT 1 AS txn_id, '2024-01-01' AS txn_date, 1000 AS amount, 'credit' AS
type
    UNION ALL SELECT 2, '2024-01-02', 200, 'debit'
    UNION ALL SELECT 3, '2024-01-03', 500, 'credit'
),
balance AS (
    SELECT txn_id, txn_date, amount, type,
           SUM(CASE WHEN type = 'credit' THEN amount ELSE -amount END)
               OVER (ORDER BY txn_date) AS running_balance
    FROM transactions
)
SELECT * FROM balance;
```

WHAT THIS QUERY DOES

Two things worth noticing here. First, the `transactions` CTE isn't built from a real table at all — it's hand-typed sample rows using `SELECT ... UNION ALL SELECT ...`, a handy way to mock up a small dataset for testing a query idea without needing to actually create and populate a table. Second, `CASE WHEN` inside

the `SUM() OVER (...)` turns debits into negative numbers before summing, so the running total correctly goes up on credits and down on debits — combining conditional logic with a window function's running-total pattern from Q149.

RESULT ON OUR SAMPLE DATA

txn_id	txn_date	amount	type	running_balance
1	2024-01-01	1,000	credit	1,000
2	2024-01-02	200	debit	800
3	2024-01-03	500	credit	1,300

3 rows returned.

Q178. Bill of materials explosion HARD

```
-- Example: Manufacturing BOM
CREATE TABLE bom (
    parent_id INT,
    component_id INT,
    quantity INT
);

WITH RECURSIVE bom_explosion AS (
    SELECT parent_id, component_id, quantity, 1 AS level
    FROM bom
    WHERE parent_id = 1  -- Start with product

    UNION ALL

    SELECT b.parent_id, b.component_id, b.quantity, be.level + 1
    FROM bom b
    JOIN bom_explosion be ON b.parent_id = be.component_id
)
SELECT * FROM bom_explosion;
```

WHAT THIS QUERY DOES

An illustrative `bom` (Bill of Materials) table has been added: product 1 is built from 1× component 2 and 4× component 3, and those components are themselves built from smaller sub-components. This is a recursive CTE with a manufacturing twist — instead of a hierarchy of *people* or *categories*, it's a hierarchy of *parts*, where each round of recursion 'explodes' one level further down into sub-components. This is exactly how manufacturing and inventory systems compute 'to build 10 finished products, how many of every individual screw, bolt, and sub-assembly do I actually need' — the same recursive-CTE skeleton as every other hierarchy question in this level, just applied to a bill of materials instead of an org chart.

RESULT ON OUR SAMPLE DATA

parent_id	component_id	quantity	level
1	2	1	1
1	3	4	1
2	4	2	2
3	5	1	2

4 rows returned.

Q179. Organization tree with depth HARD

```
WITH RECURSIVE org_tree AS (
    SELECT emp_id, name, manager_id, 0 AS depth,
           CAST(name AS CHAR(500)) AS org_path
    FROM employees
    WHERE manager_id IS NULL

    UNION ALL

    SELECT e.emp_id, e.name, e.manager_id, ot.depth + 1,
           CONCAT(ot.org_path, ' > ', e.name)
    FROM employees e
    JOIN org_tree ot ON e.manager_id = ot.emp_id
)
SELECT depth, name, org_path
FROM org_tree
ORDER BY depth, name;
```

WHAT THIS QUERY DOES

Functionally identical to Q173's employee hierarchy — same base case, same recursive case, same path-building — just renaming `level` to `depth` and finishing with an `ORDER BY` that groups the output by organizational depth rather than by employee ID. Included again here by the original cheat sheet as a second worked example of the recursive-hierarchy pattern, since it's important enough to see twice.

RESULT ON OUR SAMPLE DATA

depth	name	org_path
0	Aarav Sharma	Aarav Sharma
1	Karthik Nair	Aarav Sharma > Karthik Nair
1	Neha Gupta	Aarav Sharma > Neha Gupta
1	Priya Patel	Aarav Sharma > Priya Patel
1	Rahul Singh	Aarav Sharma > Rahul Singh
1	Ravi Teja	Aarav Sharma > Ravi Teja
1	Vikram Reddy	Aarav Sharma > Vikram Reddy

depth	name	org_path
2	Ananya Menon	Aarav Sharma > Priya Patel > Ananya Menon
2	Arjun Reddy	Aarav Sharma > Vikram Reddy > Arjun Reddy
2	Deepa Iyer	Aarav Sharma > Priya Patel > Deepa Iyer
2	Kavya Nair	Aarav Sharma > Rahul Singh > Kavya Nair
2	Meera Krishnan	Aarav Sharma > Vikram Reddy > Meera Krishnan
2	Rajesh Kumar	Aarav Sharma > Rahul Singh > Rajesh Kumar
2	Suresh Babu	Aarav Sharma > Vikram Reddy > Suresh Babu
3	Sneha Patel	Aarav Sharma > Vikram Reddy > Meera Krishnan > Sneha Patel

15 rows returned.

Q180. Folder hierarchy MEDIUM

```
-- Example: File system folders
CREATE TABLE folders (
    folder_id INT PRIMARY KEY,
    folder_name VARCHAR(100),
    parent_id INT
);

WITH RECURSIVE folder_tree AS (
    SELECT folder_id, folder_name, parent_id, 0 AS level,
           folder_name AS full_path
    FROM folders
    WHERE parent_id IS NULL

    UNION ALL

    SELECT f.folder_id, f.folder_name, f.parent_id, ft.level + 1,
           CONCAT(ft.full_path, '/', f.folder_name)
    FROM folders f
    JOIN folder_tree ft ON f.parent_id = ft.folder_id
)
SELECT * FROM folder_tree ORDER BY full_path;
```

WHAT THIS QUERY DOES

An illustrative `folders` table has been added (`Documents/Projects/SQL-Book` and a separate `Photos/2024` branch). The final worked example of this level's recursive-hierarchy family, applied to a file system: exactly the same base case / recursive case / path-building skeleton as employee hierarchies (Q173/179) and category trees (Q174), just joined with '/' instead of ' > ' to produce familiar-looking file paths. By this point in the level, recognizing 'self-referencing parent_id column + need the full ancestry path' as the signal to reach for a recursive CTE should feel like second nature.

RESULT ON OUR SAMPLE DATA

folder_id	folder_name	parent_id	level	full_path
1	Documents	*NULL*	0	Documents
2	Projects	1	1	Documents/Projects
3	SQL-Book	2	2	Documents/Projects/SQL-Book
4	Photos	*NULL*	0	Photos
5	2024	4	1	Photos/2024

5 rows returned.